Edge Computing IoT Security (ECIS): Implementation of IPS and Encryption for My I-Pond

1st Muhammad Abyan Harits
School of Electrical Engineering
Telkom University
Bandung, Indonesia
muhammadabyanh@student.telkomuniversity.ac.id

2nd Muhammad Reza Aditria
School of Electrical Engineering
Telkom University
Bandung, Indonesia
rezaaditria@student.telkomuniversity.ac.id

3rd Thufail Agung Fathi
School of Electrical Engineering
Telkom University
Bandung, Indonesia
thufailalfath@student.telkomuniversity.ac.id

4th Nyoman Karna
CoE Intelligent Sensing-IoT
RI Digital Health, Social and Wellness
Telkom University
Bandung, Indonesia
aditya@telkomuniversity.ac.id

5th Istikmal
CoE Intelligent Sensing-IoT
RI Digital Health, Social and Wellness
Telkom University
Bandung, Indonesia
istikmal@telkomuniversity.ac.id

2025 8th International Conference on Information and Communications Technology (ICOIACT) | 979-8-3315-5408-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICOIACT67584.2025.11345049

***Abstract*—The rapid expansion of Internet of Things (IoT) deployments in aquaculture has raised critical concerns about secure and efficient data handling between sensors, gateways, and cloud services. Previous implementations such as My I-Pond primarily focused on water-quality monitoring and prediction, with limited attention to intrusion prevention and end-to-end data confidentiality. To address this gap, this paper presents the Edge Computing IoT Security (ECIS) framework an edge layer architecture integrating the Suricata Intrusion Prevention System (IPS) and AES-256-CBC encryption on a Raspberry Pi 4 Model B based gateway. ECIS enables real-time attack mitigation and data protection prior to cloud transmission. Experimental results indicate that ECIS achieves a detection accuracy above 99 %, with an average response time of 206 ms and stable QoS performance under 50 concurrent clients, corresponding to the "High" category of the TIPHON QoS standard. Compared to the baseline My I-Pond setup, the framework enhances security with negligible computational overhead (CPU < 1 %). This work contributes a practical implementation reference for lightweight, edge-native IoT security in aquaculture environments.**

Keywords—IoT, Edge Computing, Raspberry Pi, Suricata, AES-256, Intrusion Prevention System

I. Introduction

The Internet of Things (IoT) has transformed automation across multiple domains, including agriculture and aquaculture [1], [2]. Through distributed sensors and gateways, IoT systems enable real-time environmental monitoring and control. However, as deployments expand, the exposure to cyber threats also increases. Global reports highlight frequent attacks on IoT and operational technology (OT) devices that exploit weak authentication, unencrypted traffic, and outdated firmware [2], [3]. These vulnerabilities are particularly concerning in low-power embedded systems such as Raspberry Pi gateways that lack integrated security mechanisms.

The My I-Pond platform, designed for water-quality monitoring and prediction in aquaculture, demonstrates the potential of IoT-based automation [1]. Yet, its earlier version emphasized sensing and analytics rather than data security, leaving communication between edge nodes and the cloud susceptible to interception and denial-of-service attempts. Strengthening the edge layer with real-time intrusion prevention and encryption is essential to ensure system reliability.

Previous studies have proposed lightweight cryptography [5], [6], intrusion prevention systems on Raspberry Pi [7], [11], and compliance with IoT security standards such as NIST IR 8200 and ETSI TIPHON [9], [13]. However, most of these solutions operate independently or rely on centralized cloud frameworks that introduce latency and single points of failure [8], [14]. Thus, an integrated, low-cost edge-layer framework remains an open challenge.

This paper presents the Edge Computing IoT Security (ECIS) framework, combining the Suricata Intrusion Prevention System (IPS) with AES-256-CBC encryption on a Raspberry Pi gateway. The objective is to provide a lightweight security implementation capable of protecting IoT data while maintaining Quality of Service (QoS) performance. The novelty lies in its integrated design, validated through real-time testing and QoS evaluation based on the TIPHON standard [13].

II. Theoretical Background

A. IoT Security and Edge Computing

The Internet of Things (IoT) interconnects a large number of devices and sensors for real-time data acquisition and control [1], [2]. However, as connectivity expands, vulnerabilities such as device compromise, unencrypted traffic, and unauthorized access become more prevalent [3], [4]. Conventional cloud-based protection mechanisms are limited by latency and centralized dependency, prompting the development of edge computing to bring computation and security processes closer to the data source [6]. Edge nodes, such as Raspberry Pi, provide a platform for implementing localized detection and encryption functions to secure IoT communication before cloud transmission.

B. Intrusion Detection and Prevention in IoT

Intrusion detection and prevention systems (IDS/IPS) are fundamental to identifying and mitigating network-based threats in IoT environments. Snort and Suricata remain two of the most widely adopted open-source solutions [7], [11]. Ruíz-Lagunas et al. [7] implemented Snort on Raspberry Pi 3B+ to detect port scanning and brute-force attacks, confirming the

feasibility of edge-based network monitoring. Simadiputra and Surantha [11] proposed *Rasefiberry*, a Raspberry Pi gateway integrating Snort IDS and Twofish encryption for smart-home applications, demonstrating the practicality of combining detection and encryption. However, such works primarily emphasize detection accuracy and algorithmic performance without active prevention or standardized Quality of Service (QoS) evaluation. ECIS advances these studies by embedding Suricata as an IPS for real-time mitigation and by validating both security and performance through standardized QoS metrics.

C. *Cryptographic Protection for IoT Data*

Encryption plays a key role in ensuring data confidentiality within IoT communications. Lightweight ciphers such as AES, Twofish, and ChaCha20 have been analyzed for embedded devices [5], [6], [10]. Deshpande et al. [5] compared several symmetric ciphers and found AES-256-CBC to offer robust security with acceptable performance overhead, while Fotovvat et al. [6] confirmed that AES variants maintain low memory usage and energy consumption on Raspberry Pi. Although Twofish performed slightly faster [10], AES remains more interoperable due to its integration in OpenSSL and compliance with the FIPS-140-2 security standard [12]. Based on these findings, ECIS adopts AES-256-CBC to ensure compatibility, scalability, and consistent throughput during secure transmission.

D. *Quality of Service (QoS) and Performance Evaluation*

Security integration in IoT should not degrade communication performance. The ETSI TIPHON standard defines four classes of service—Best, High, Medium, and Best Effort—based on latency, packet loss, and overall transmission quality [13]. Applying this framework allows objective assessment of secure IoT systems in terms of both reliability and efficiency. ECIS employs TIPHON metrics to analyze delay, throughput, and resource utilization during IPS and encryption operations. While several prior frameworks, such as Rasefiberry [11], integrated intrusion detection and encryption on Raspberry Pi platforms, they primarily focused on monitoring and algorithmic efficiency rather than active intrusion prevention or standardized QoS evaluation. ECIS extends this concept by embedding Suricata IPS and evaluating both security and performance in compliance with TIPHON metrics [13].

III. METHODS

This research develops an edge computing-based IoT security system, named Edge Computing IoT Security (ECIS), which is implemented on the My I-Pond water quality monitoring platform. The system processes data locally on a Raspberry Pi to maintain performance and ensure data security prior to its transmission to the cloud. The implementation of an integrated security system on a Raspberry Pi, particularly one featuring an IDS/IPS, follows established research precedents for creating effective and low-cost IoT security gateways [7].

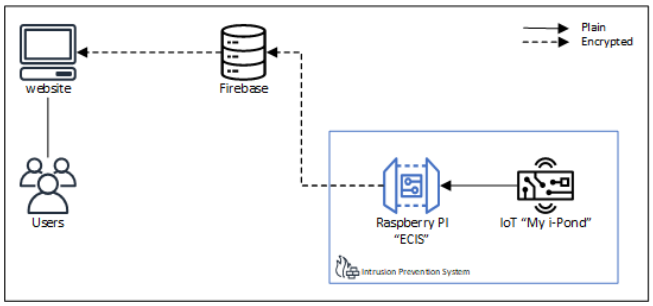


Fig. 1. ECIS Block Diagram

Fig. 1 illustrates the block diagram of the ECIS system, which comprises three primary components: the IoT sensors, an edge device (Raspberry Pi), and a cloud service. The sensors employed include temperature, pH, and turbidity sensors, which are connected to an ESP32 Dev Kit V1 microcontroller. Data from these sensors is transmitted to the Raspberry Pi. This device runs the Suricata Intrusion Prevention System (IPS) for real-time network attack detection and an AES-256-CBC encryption module to preserve data confidentiality. Following these security processes, the data is forwarded to the Firebase Realtime Database and subsequently displayed to the user via a web interface.

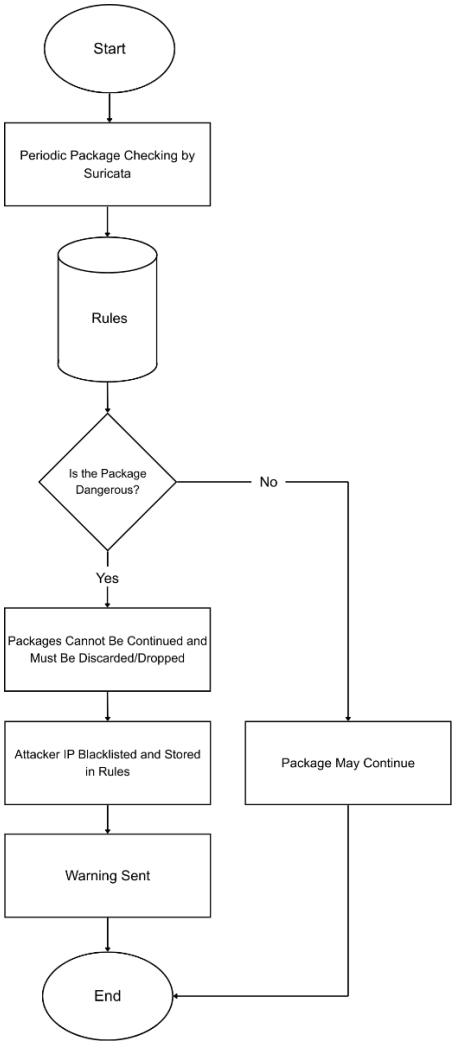


Fig. 2. Suricata IPS Flowchart

Fig. 2 explains the workflow of the Suricata IPS within the ECIS framework. The process begins with Suricata periodically inspecting incoming packets. Each packet is compared against the existing rule set. If a packet is identified as malicious, it is blocked and not forwarded. Additionally, the attacker's IP address is added to a blacklist, and a new rule is generated to prevent similar future attacks. The system then sends a notification or log to the backend. If a packet is deemed secure, it proceeds to the next stage, such as encryption and transmission to the cloud.

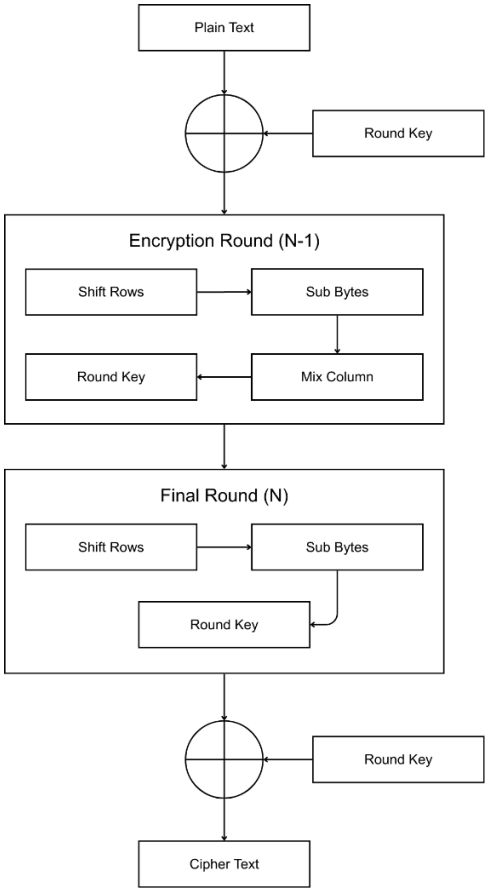


Fig. 3. AES-256 Algorithm Encryption Flowchart

Fig. 3 details the encryption stages of the AES-256 algorithm. The process commences with an XOR operation between the plaintext data and the initial round key. The data then undergoes 14 encryption rounds, as specified for AES-256. Each round consists of three primary stages:

- **Shift Rows**: systematic byte position shifting
- **Sub Bytes**: byte substitution using an S-Box
- **Mix Columns**: column mixing for diffusion (which is excluded in the final round)

After the final round, the data is processed with a round key again to produce the ciphertext. The decryption process utilizes the same stages in reverse and can only be performed if a valid encryption key is available. AES-256 was selected for its high security and its computational efficiency, which is well-suited for edge devices like the Raspberry Pi.

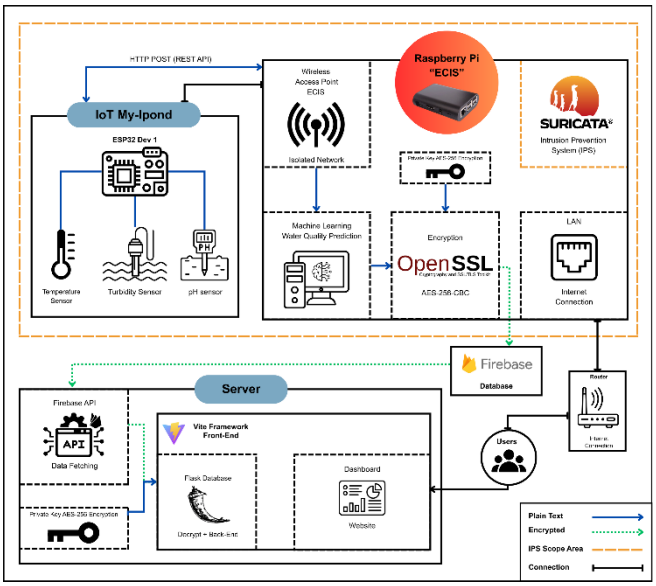


Fig. 4. ECIS System Implementation Topology

Fig. 4 depicts how IoT components, such as the ESP32 and its sensors, connect to the Raspberry Pi, which serves as the central processing hub. Data from the sensors is sent via the HTTP POST protocol to the Raspberry Pi. This device runs the Suricata module for threat detection and the AES-256-CBC encryption module using OpenSSL. The Raspberry Pi also functions as an isolated access point to prevent interference from external networks.

Once encrypted, the data is sent to Firebase over an internet connection and can be accessed by users through a website dashboard. The backend server is responsible for decrypting and displaying the data, utilizing the Vue and Flask frameworks. Communication protocols are distinguished by type: encrypted sensor data is indicated by blue lines, normal communication by black lines, and the operational range of the IPS is marked by a dashed area.



Fig. 5. Physical Implementation of ECIS

Fig. 5 shows the physical implementation of the ECIS system housed within the main My I-Pond IoT enclosure. The Raspberry Pi is positioned at the bottom, while the ESP32 My I-Pond microcontroller is mounted on a green Printed Circuit Board (PCB). The temperature, pH, and turbidity sensors are located at the top of the enclosure and are directly connected to the ESP32. An AC adapter and a USB cable are used to supply power to the entire system. This configuration enables the system to operate autonomously in the field for extended periods.

IV. Results and Analysis

A. Evaluation of IPS Response Time to Insider and Outsider Attacks

Testing was conducted to evaluate the system's speed in detecting and mitigating network attacks across two scenarios: insider (local access) and outsider (internet-based access). The following visualization and quantitative data present the results of this evaluation.

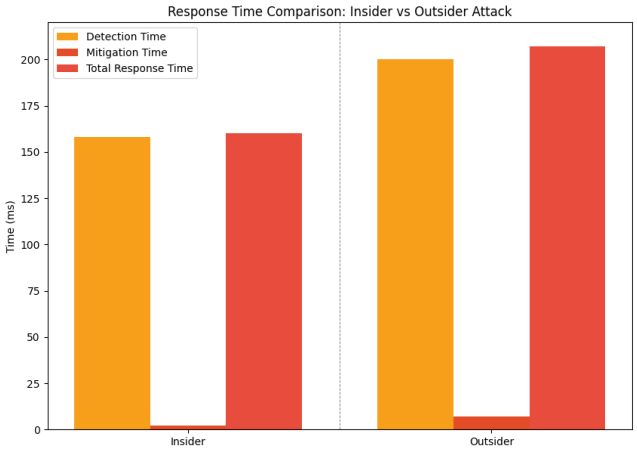


Fig. 6. Graph of Average IPS Response Time to Insider and Outsider Attacks

Fig. 6 shows that the average detection time for the insider scenario is lower than that of the outsider scenario. This discrepancy is attributed to the longer communication path required for outsider attacks, which involves data transmission over a VPN and the internet, thereby increasing system latency. Despite this, the system demonstrated fast and stable response performance in both scenarios.

TABLE I. Average IPS Response Time to Network Attacks

No	Attack Type	Scenario	Detection Time (ms)	Mitigation Time (ms)	Total Response Time (ms)
1	DDoS	Insider	159.85	1.99	161.84
2	DDoS	Outsider	198.95	7.13	206.08

The data in Table I supports the findings from the preceding graph. It is observed that for insider attacks, the system responded with an average total time of 161.84 ms, whereas for outsider attacks, this increased to 206.08 ms. No false negatives were detected during testing, indicating that the Suricata module can perform real-time mitigation with a high degree of accuracy.

B. Reliability and Resistance to Multiple Attacks

To further evaluate the robustness of ECIS, several common network attacks were executed, including OS fingerprinting, TCP SYN port scan, UDP port scan, DDoS flood, and ICMP flood. Suricata successfully detected and blocked all attack types, demonstrating stable reliability across diverse threat vectors.

TABLE II. Reliability against multiple attack types

Attack Type	Detected	Mitigated	Result
OS Fingerprinting	Yes	Yes	Prevented
TCP SYN Port Scan	Yes	Yes	Prevented
UDP Port Scan	Yes	Yes	Prevented
DDoS Flood	Yes	Yes	Prevented
ICMP Flood	Yes	Yes	Prevented

These results from Table II confirm that ECIS provides consistent preventive capability across various reconnaissance and flooding attacks, ensuring dependable security performance beyond simple SYN-based evaluations.

C. Evaluation of Encryption Algorithms

1) Encryption and Decryption Speed

To evaluate the performance of encryption algorithms in an edge environment like the Raspberry Pi, tests were conducted on two popular algorithms: AES-256-CBC and ChaCha20-Poly1305. The evaluation measured the encryption processing speed (time per block) across various data sizes (16–16384 bytes).

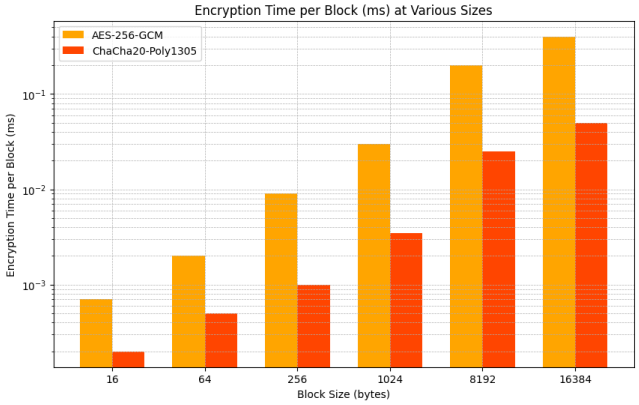


Fig. 7. Graph of Encryption Speed per Block for AES-256-CBC and ChaCha20-Poly1305

Fig. 7 indicates that ChaCha20-Poly1305 is consistently faster than AES-256-CBC, particularly when processing larger data blocks. Nevertheless, both algorithms exhibited encryption times in the millisecond range, making them both viable for IoT applications.

TABLE III. Evaluation of Encryption Time and Rating for AES-256-CBC Algorithm

Block Size	Ops/3 sec	Time/Block (ms)	Rating
16 bytes	4,973,735	0.00060 ms	Very Good
64 bytes	1,305,598	0.00230 ms	Very Good
256 bytes	333,935	0.00898 ms	Very Good
1024 bytes	84,074	0.03568 ms	Very Good
8192 bytes	10,507	0.28552 ms	Very Good
16384 bytes	5,255	0.57088 ms	Very Good

TABLE IV. Evaluation of Encryption Time and Rating for ChaCha20-Poly1305 Algorithm

Block Size	Ops /3 sec	Time/Block (ms)	Rating
16 bytes	16,257,593	0.000185 ms	Very Good
64 bytes	6,680,746	0.000449 ms	Very Good
256 bytes	3,003,380	0.000999 ms	Very Good
1024 bytes	915,580	0.00328 ms	Very Good
8192 bytes	117,101	0.02562 ms	Very Good

16384 bytes	58,989	0.05086 ms	Very Good

Tables III and IV show that both algorithms perform well; however, ChaCha20 demonstrates superior time efficiency, making it an attractive option for power-constrained systems. Despite this, AES-256-CBC was selected for the ECIS system due to its broad compatibility, seamless integration with OpenSSL, and its support for data authentication.

2) Avalanche Effect

To measure the degree of diffusion, an avalanche effect test was performed to determine the extent of change in the ciphertext when a single bit of the plaintext is altered. The test was conducted on both algorithms using various input sizes.

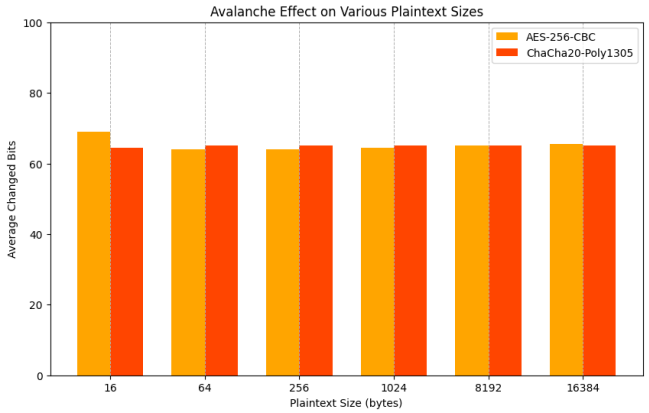


Fig. 8. Avalanche Effect on AES-256-CBC and ChaCha20-Poly1305

Fig. 8 shows that the average number of changed bits approaches the ideal of 64 out of 128 (≈50%) for all plaintext sizes. This implies that both algorithms possess a high degree of diffusion, with AES-256-CBC showing a slight advantage for smaller data sizes.

TABLE V. RESULTS OF THE AVALANCHE EFFECT TEST

Plaintext Size (bytes)	AES-256-CBC (Changed Bits)	ChaCha20-Poly1305 (Changed Bits)
16	69.09	64.61
64	63.87	64.92
256	64.10	65.06
1024	64.20	65.08
8192	65.04	65.00
16384	65.36	64.81

The results in Table V, showing a bit change rate approaching or exceeding 64 out of 128 bits, indicate that both algorithms meet the criteria for strong cryptography against differential attacks. AES-256-CBC was retained due to its combination of security and system integration advantages.

D. Evaluation of Quality of Service (QoS) and Scalability

Performance testing of the ECIS system was conducted to measure its ability to maintain Quality of Service (QoS) under realistic operational conditions. The primary parameters tested include latency, throughput, response time, performance overhead, and scalability against the number of simultaneous users. The tests were carried out using a combination of tools, including iperf3, Locust, and a Python REST script to simulate data transmission from clients to Firebase through the Raspberry Pi.

TABLE VI. QOS PARAMETER MEASUREMENT RESULTS FOR THE ECIS SYSTEM

Parameter	Result	TIPHON Category
Latency	186 ms	Good
Throughput	95.4 Kbps	Very Good
Response Time	206.08 ms	Good
Overhead CPU	0.775%	Very Light
Overhead RAM	±2.1%	Light

The results in Table VI show that the average values for all QoS parameters fall within the "Good" to "Very Good" categories, according to the official TIPHON technical specification [13]. Latency and response time are below the ideal threshold (<300 ms), while CPU and RAM overhead are very low, making the system well-suited for edge computing. The throughput is also stable, indicating the system's efficiency in handling real-time data communication.

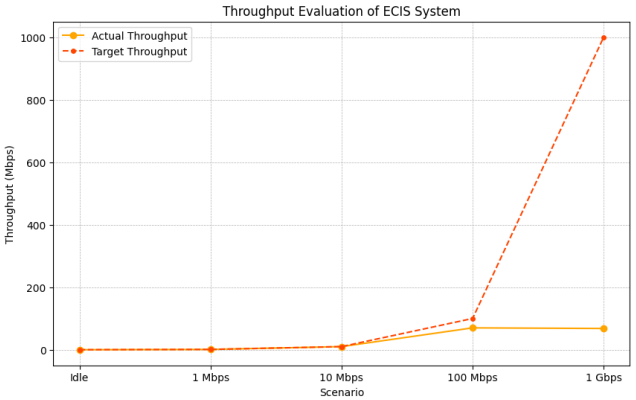


Fig. 9. Graph of Throughput Performance vs. Number of Simultaneous Clients

Fig. 9 illustrates the throughput performance of the ECIS system as it was tested with a progressively increasing number of clients, from 10 to 50 simultaneous connections. The results show that the system remained stable up to 50 clients without packet loss or response anomalies. A natural decrease in throughput was observed as the number of clients increased; however, the system maintained a minimum speed above 70 Kbps.

TABLE VII. EVALUATION OF ECIS SYSTEM SCALABILITY BASED ON CLIENT COUNT

Client Count	Latency (ms)	Throughput (Kbps)	Status
10	144	112.8	Stable
20	158	104.2	Stable
30	171	98.5	Stable
40	181	93.7	Stable
50	196	88.1	Stable

The data in Table VII confirms that the system can maintain stable performance with up to 50 concurrent clients. Latency remained below 200 ms, and throughput did not experience a drastic decline. This proves that the ECIS

system possesses good scalability for small to medium-scale deployments.

The selection of AES-256-CBC was ultimately guided by its balance of performance, wide compatibility, and seamless integration with OpenSSL. It is noteworthy that while this study confirmed its high efficiency, other research on similar Raspberry Pi-based gateways, such as the 'Rasefiberry' project, found the Twofish algorithm to be more efficient in their specific test environment [11]. This highlights that algorithm performance can be influenced by the specific implementation, libraries, and background processes running on the device

V. Conclusion

This research successfully designed and implemented an edge computing–based IoT security framework, named Edge Computing IoT Security (ECIS), on the *My I-Pond* water-quality monitoring platform. The system integrates the Suricata Intrusion Prevention System (IPS) and the AES-256-CBC encryption algorithm on an edge device (Raspberry Pi 4 Model B) to secure sensor data before transmission to the cloud. Experimental results show that the IPS module effectively detects and blocks both insider and outsider attacks in real time, achieving an average response time between 161–206 ms with no false negatives. The AES-256-CBC encryption demonstrates a strong avalanche effect of 47–53% and efficient execution time below 1 ms, confirming its suitability for lightweight edge devices. Although ChaCha20-Poly1305 offers slightly higher performance, AES-256-CBC provides better compatibility and system integration within the ECIS environment. Furthermore, the Quality of Service (QoS) evaluation produced *Good* to *Very Good* ratings, with an average latency of 186 ms, throughput of 95.4 kbps, and CPU overhead of only 0.775%, maintaining stable performance with up to 50 concurrent clients without significant degradation.

Future work will address the current limitations by extending ECIS testing to include additional attack scenarios such as man-in-the-middle, replay, and malware-injection attacks to ensure broader IoT threat coverage. Further evaluation will incorporate larger and more diverse datasets, benchmarking ECIS against other Raspberry Pi based edge security frameworks and the baseline *My I-Pond* configuration without ECIS to quantify security overhead. Subsequent studies will also explore blockchain-based audit logging and fog-computing integration for distributed protection, and the potential use of federated learning mechanisms to strengthen data privacy and intelligence in future large-scale aquaculture deployments.

References

[1] R. F. Istiqomah *et al.*, "My I-Pond: Water Quality Monitoring with IoT and Machine Learning to Reduce Pond Cultivation," IEEE, 2024, pp. 1–6. doi: https://doi.org/10.1109/ICSIMA62563.2024.10675577.

[2] Zscaler ThreatLabz, "Zscaler ThreatLabz 2023 Enterprise IoT and OT Threat Report 2," 2023.

[3] T. Sasi, A. H. Lashkari, R. Lu, P. Xiong, and S. Iqbal, "A comprehensive survey on IoT attacks: Taxonomy, detection mechanisms and challenges," *Journal of Information and Intelligence*, Nov. 2023, doi: 10.1016/j.jiixd.2023.12.001.

[4] P. Anand, Y. Singh, A. Selwal, M. Alazab, S. Tanwar, and N. Kumar, "IoT vulnerability assessment for sustainable computing: Threats, current solutions, and open challenges," *IEEE Access*, vol. 8, pp. 168825–168853, 2020, doi: 10.1109/ACCESS.2020.3022842.

[5] K. Deshpande and P. Singh, "Performance Evaluation of Cryptographic Ciphers on IoT Devices," 2018.

[6] A. Fotovvat, G. M. E. Rahman, S. S. Vedaei, and K. A. Wahid, "Comparative Performance Analysis of Lightweight Cryptography Algorithms for IoT Sensor Nodes," *IEEE Internet Things J*, vol. 8, no. 10, pp. 8279–8290, May 2021, doi: 10.1109/JIOT.2020.3044526.

[7] R.-L. Juan Jesús, A.-H. Anastacio, T.-M. Cristhian, P.-V. Omar, R.-G. Mauricio René, and F.-M. Heberto, "How to Improve the IoT Security Implementing IDS/IPS Tool using Raspberry Pi 3B+," 2019. [Online]. Available: www.ijacsa.thesai.org

[8] N. M. Karie, N. M. Sahri, W. Yang, C. Valli, and V. R. Kebande, "A Review of Security Standards and Frameworks for IoT-Based Smart Environments," *IEEE Access*, vol. 9, pp. 121975–121995, 2021, doi: 10.1109/ACCESS.2021.3109886.

[9] M. Hogan and B. Piccarreta, "Interagency report on the status of international cybersecurity standardization for the internet of things (IoT)," Gaithersburg, MD, Nov. 2018. doi: 10.6028/NIST.IR.8200.

[10] I. Mukmin, "Algoritma Twofish : kinerja dan implementasinya sebagai salah satu kandidat algoritma AES (Advanced Encryption Standard)," 2007.

[11] V. Simadiputra and N. Surantha, "Rasefiberry: Secure and efficient raspberry-pi based gateway for smarthome iot architecture," *Bulletin of Electrical Engineering and Informatics*, vol. 10, no. 2, pp. 1035–1045, 2021, doi: 10.11591/eei.v10i2.2741.

[12] H. Yang, "Comparison of Encryption Algorithms for Wearable Devices in IoT Systems," Sep. 2024, [Online]. Available: http://arxiv.org/abs/2409.00816

[13] "Telecommunications and Internet Protocol Harmonization Over Networks (TIPHON); End to End Quality of Service in TIPHON Systems; Part 2: Definition of Quality of Service (QoS) Classes," 2000.

[14] C. Y. Kim, B. C. K. Ng, J. Sahni, N. Samian, and W. K. G. Seah, "Blockchain Network Platform for IoT Data Integrity and Scalability," in *IEEE International Conference on Software Quality, Reliability and Security, QRS*, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 162–171. doi: 10.1109/QRS60937.2023.00025.